



BINV1053

4.1 Données, fichiers et gestion des erreurs internes

Jérôme Plumat

Haute École Léonard de Vinci

2026





- 1 Introduction
- 2 Rappels
- 3 Erreur interne
- 4 Les données du serveur
- 5 DBO
- 6 Le service de fichiers
- 7 Les services ressources



1. Introduction

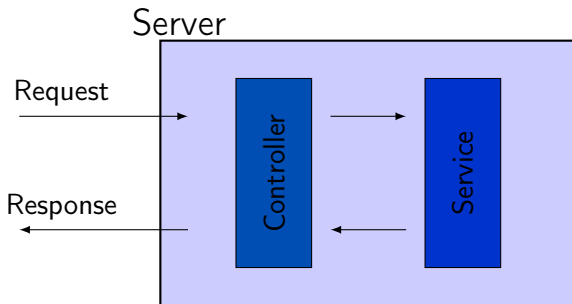


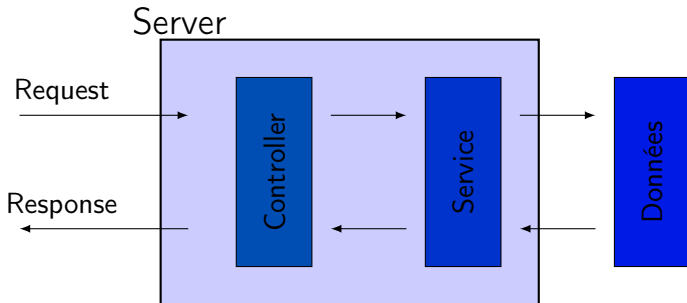
Objectifs

- Comprendre la gestion des données et leur stockage dans des fichiers.
- Pouvoir utiliser un service permettant de lire et d'écrire des données dans un fichier.
- Pouvoir créer un service qui gère une ressource.
- Comprendre et utiliser des fonctions génériques.
- Pouvoir gérer les erreurs lors de la lecture et de l'écriture des données dans un fichier.



2. Rappels







3. Erreur interne



Nous nommerons **erreur interne** (internal error en anglais) des erreurs générées par nos services et indépendantes de la requête émise par le client. Ces erreurs sont typiquement dues à une base de données, un fichier, ou une ressource externe. Cette ressource peut être indisponible, ou inaccessible, ou encore mal configurée.

Ces erreurs sont attrapées dans le bloc `try/catch` dans nos services et loggées sur le niveau `error` de notre logger.



4. Les données du serveur



Notre serveur est un élément informatique qui permet de stocker et de fournir des données.

A ce stade, notre serveur ne peut retenir aucune donnée entre deux redémarrages du serveur car elles sont écrites en dur dans notre code (elles sont **statiques**).

Les données du serveur

Dans la réalité



Un vrai serveur possède une base de données qui permet de stocker des données de manière **dynamique**.

Dans une mise en production réaliste, une base de données (par exemple SQL) serait déployée sur un autre serveur. Notre serveur enverrait des requêtes SQL à ce serveur pour effectuer des opérations sur la base de données.

Les données du serveur

Dans notre cours



Dans le cadre de ce cours, nous utiliserons des fichiers pour stocker des données de manière dynamique.

Cette solution nous permettra de coder rapidement un serveur indépendant. Nous nous appliquerons à développer une solution qui permettrait de déployer rapidement une base de données si cela s'avérait nécessaire.



5. DBO

Les données du serveur

Les différentes représentations du modèle



Jusqu'à présent nous avons utilisé des objets représentant nos ressources (par exemple patient, doctor, etc.).

Nous avons désigné par "modèle" (ou "model") ces objets. **Nous appellerons "modèles" ce qui vit au sein de notre serveur.**

Durant le développement d'une application, il est important de distinguer les objets qui vivent au sein de notre serveur et les objets qui viennent des autres acteurs (par exemple le client, la base de données, etc.).

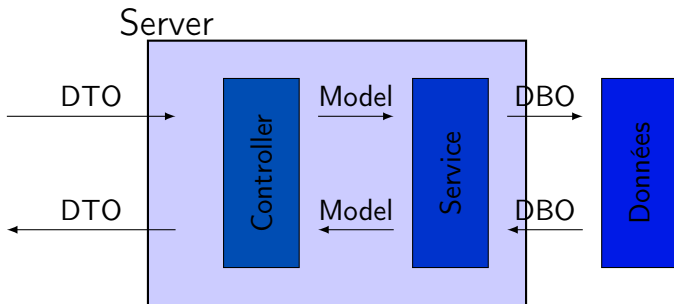


On distinguera en particulier :

- **Un modèle** : une représentation d'une ressource qui vit au sein de notre serveur.
- **Un DBO** (Data Base Object) : une ressource qui provient du système de stockage des données ou qui est envoyée à celui-ci.
- **Un DTO** (Data Transfer Object) : une ressource qui provient du client ou qui est envoyée à celui-ci.

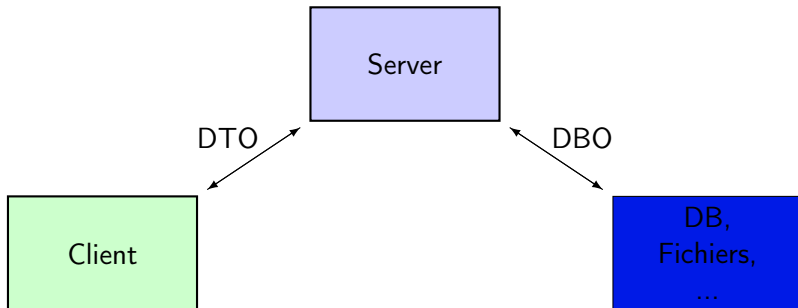
DBO

Les différents types d'objets



Les données du serveur

Les différents modèles





6. Le service de fichiers



Le `FileService` est une classe qui permet d'interagir avec des fichiers. Il permet de lire et d'écrire des données dans un fichier.

Elle possède deux méthodes principales :

- `readFile` : pour lire un fichier.
- `writeFile` : pour écrire dans un fichier.



Le service est implémenté comme une classe ne contenant que des fonctions statiques. Le fichier `services/files.service.ts` contient le code suivant :

```
1  public class FileService {
2      public static readFile<T>(path: string): T {
3          // ...
4      }
5
6      public static writeFile<T>(path: string, data: T) {
7          // ...
8      }
9  }
```



On remarque que les méthodes de ce service sont **génériques**.

Génériques

Une fonction générique est une fonction dont on spécifie le type des données lors de son appel.

Ainsi, la fonction `readFile<T>(path: string): T` permet de lire un fichier et de retourner des données de type `T`. Le type `T` est spécifié lors de l'appel de la fonction. Par exemple

```
1 import { DoctorDBO } from "../models/doctor.model";  
2 // ...  
3 const data : DoctorDBO[] = FileService.readFile<DoctorDBO>("../data/  
  doctors.json");
```



Attention

Les fonctions `readFile` et `writeFile` sont des fonctions qui peuvent lever des erreurs. Il faut donc toujours les utiliser dans un bloc `try/catch`.

```
1 // in doctors.service.ts
2 getAll() : Doctor[] {
3   let doctorsDBO : DoctorDBO[] = [];
4   try {
5     doctorsDBO = FilesService.readFile<DoctorDBO>("../data/doctors.json");
6   } catch (error) {
7     logger.error(error);
8     return []; // some default value, depends on the context
9   }
10  const doctors : Doctor[] = [];
11  for (const dbo of doctorsDBO) {
12    doctors.push(DoctorsMapper.fromDBO(dbo));
13  }
14  return doctors;
15 }
```



7. Les services ressources



Un **service ressource** accède à une ressource dans le système de stockage (DB, fichier, etc.). Il est responsable de la gestion de cette ressource. Il effectue les opérations liées au CRUD (Create, Read, Update, Delete) sur cette ressource et **uniquement** sur celle-ci.

Il s'agit d'une classe possédant une série de méthodes **statiques** qui permettent de faire des opérations sur la ressource : `getAll`, `getById`, `create`, `update` et `delete`.



Les services ressources feront la liaison entre le contrôleur et le stockage des données. Ils seront appelés par les contrôleurs lors du traitement d'une requête, ils les traiteront et renverront les données placées dans la réponse.

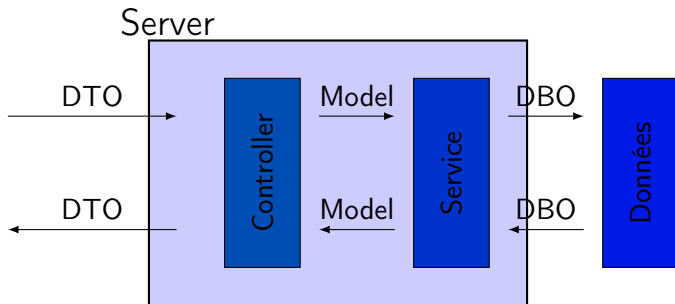
Les services traiteront avec les DBO et ne retourneront que des modèles.



Abstraction

Les services gérant les différentes ressources permettent de cacher les détails de la gestion des données et des erreurs.

En particulier, ils exposent des fonctions qui n'acceptent et retournent **uniquement** des objets de type modèle (`Doctor`, `Patient`, etc.).



Les services ressources

Différents types de services

